FINAL SYSTEM DOCUMENTATION

STUDENT INFORMATION MANAGEMENT SYSTEM

name by: Elizaga Christopher Julius P.

Grade/Section: BSIT-1B

Subject:COC101/final system

Instructor: Samuel Linda Ronstadt Jr.

Date Submitted: 011/2/26

1. Introduction

The Student Information Management System is a simple Java console application designed to store and display basic student information. It helps a user enter student names, student numbers, courses, and sections, then view the records in one place.

2. Objectives

- To record basic student information.
- To display saved records in an organized format.
- To practice Java variables, arrays, loops, switch statements, and if-else conditions.

3. Scope and Limitations

Scope: This version allows the user to add student records during one program session, display all records, and exit the program.

Limitations: Records are stored in memory only and disappear when the program closes. The system does not use a database, login, or edit/delete feature.

4. System Requirements

Hardware: computer or laptop. Software: Java JDK and IntelliJ IDEA or another Java IDE.

5. System Design

Algorithm: Start → Show menu → Read choice → Add student, display students, or exit → Repeat until Exit → End.

Program structure: The program imports Scanner, declares the required variables or arrays, displays a menu or prompts, processes input, validates user choices, and prints the result.

6. Source Code

```
import java.util.Scanner;

public class StudentInformationSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] names = new String[100];
        String[] numbers = new String[100];
```

```
String[] courses = new String[100];
int count = 0;
int choice;

do {
    System.out.println("\n=== STUDENT INFORMATION SYSTEM ===");
    System.out.println("1. Add Student");
    System.out.println("2. Display Students");
    System.out.println("3. Exit");
    System.out.print("Choose: ");
    choice = sc.nextInt();
    sc.nextLine();

    switch (choice) {
        case 1:
            if (count < names.length) {
                System.out.print("Student number: ");
                numbers[count] = sc.nextLine();
                System.out.print("Student name: ");
                names[count] = sc.nextLine();
                System.out.print("Course/Strand and section: ");
                courses[count] = sc.nextLine();
                count++;
                System.out.println("Student record added.");
            } else {
                System.out.println("Record limit reached.");
            }
            break;
        case 2:
            if (count == 0) {
                System.out.println("No student records yet.");
            } else {
                for (int i = 0; i < count; i++) {
                    System.out.println("\nRecord " + (i + 1));
                    System.out.println("Number: " + numbers[i]);
                    System.out.println("Name: " + names[i]);
                    System.out.println("Course/Section: " + courses[i]);
                }
            }
            break;
        case 3:
            System.out.println("Thank you.");
            break;
        default:
            System.out.println("Invalid choice. Try again.");
    }
} while (choice != 3);
sc.close();
    }
}
```

7. Program Screenshots

Insert your own screenshots here after running the program in IntelliJ IDEA:

Screenshot 1: Program menu or opening screen

[Paste screenshot here]

Screenshot 2: Sample input and output

[Paste screenshot here]

Screenshot 3: Validation or final result

[Paste screenshot here]

8. Testing and Results

Test	Expected Result	Actual Result
Valid input	System processes input and shows the correct output.	To be completed after testing.
Invalid menu choice or number	System displays an error message.	To be completed after testing.
Exit option	Program closes normally.	To be completed after testing.

9. Conclusion and Recommendations

The Student Information Management System demonstrates how Java can be used to solve a basic information-processing task. It applies input and output, variables, conditional statements, and other beginner programming concepts. In the future, the system may be improved by adding data storage, a graphical user interface, stronger input validation, and additional functions.

Prepared by: _____

Checked by: _____

FINAL SYSTEM DOCUMENTATION

LIBRARY MANAGEMENT SYSTEM

Prepared by: _____

Grade/Section: _____

Subject: Java Programming

Instructor: _____

Date Submitted: _____

1. Introduction

The Library Management System is a Java console application that keeps a simple list of books and tracks whether each book is available or borrowed.

2. Objectives

- To list books in a small library collection.
- To allow a user to borrow and return a book.
- To use arrays, loops, switch statements, and if-else conditions.

3. Scope and Limitations

Scope: The program starts with a small sample collection of books. The user can display books, borrow a book by its number, and return a borrowed book.

Limitations: The book list is fixed in the source code. Changes are not saved after the program closes. It does not include member accounts, due dates, or fines.

4. System Requirements

Hardware: computer or laptop. Software: Java JDK and IntelliJ IDEA or another Java IDE.

5. System Design

Algorithm: Start → Display menu → Choose list, borrow, return, or exit → Validate book number and availability → Update status → Repeat → End.

Program structure: The program imports Scanner, declares the required variables or arrays, displays a menu or prompts, processes input, validates user choices, and prints the result.

6. Source Code

```
import java.util.Scanner;

public class LibraryManagementSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] books = {"Noli Me Tangere", "El Filibusterismo", "Java Basics",
                          "World History", "Science Guide"};
        boolean[] borrowed = new boolean[books.length];
        int choice;
```

```
do {
    System.out.println("\n=== LIBRARY MANAGEMENT SYSTEM ===");
    System.out.println("1. Display Books");
    System.out.println("2. Borrow Book");
    System.out.println("3. Return Book");
    System.out.println("4. Exit");
    System.out.print("Choose: ");
    choice = sc.nextInt();

    switch (choice) {
        case 1:
            for (int i = 0; i < books.length; i++) {
                String status = borrowed[i] ? "Borrowed" : "Available";
                System.out.println((i + 1) + ". " + books[i] + " - " + status);
            }
            break;
        case 2:
            System.out.print("Enter book number: ");
            int borrowNo = sc.nextInt();
            if (borrowNo >= 1 && borrowNo <= books.length) {
                if (!borrowed[borrowNo - 1]) {
                    borrowed[borrowNo - 1] = true;
                    System.out.println("Book borrowed successfully.");
                } else {
                    System.out.println("This book is already borrowed.");
                }
            } else {
                System.out.println("Invalid book number.");
            }
            break;
        case 3:
            System.out.print("Enter book number: ");
            int returnNo = sc.nextInt();
            if (returnNo >= 1 && returnNo <= books.length) {
                if (borrowed[returnNo - 1]) {
                    borrowed[returnNo - 1] = false;
                    System.out.println("Book returned successfully.");
                } else {
                    System.out.println("This book was not borrowed.");
                }
            } else {
                System.out.println("Invalid book number.");
            }
            break;
        case 4:
            System.out.println("Goodbye.");
            break;
        default:
            System.out.println("Invalid choice.");
    }
} while (choice != 4);
sc.close();
}
```

7. Program Screenshots

Insert your own screenshots here after running the program in IntelliJ IDEA:

Screenshot 1: Program menu or opening screen

[Paste screenshot here]

Screenshot 2: Sample input and output

[Paste screenshot here]

Screenshot 3: Validation or final result

[Paste screenshot here]

8. Testing and Results

Test	Expected Result	Actual Result
Valid input	System processes input and shows the correct output.	To be completed after testing.
Invalid menu choice or number	System displays an error message.	To be completed after testing.
Exit option	Program closes normally.	To be completed after testing.

9. Conclusion and Recommendations

The Library Management System demonstrates how Java can be used to solve a basic information-processing task. It applies input and output, variables, conditional statements, and other beginner programming concepts. In the future, the system may be improved by adding data storage, a graphical user interface, stronger input validation, and additional functions.

Prepared by: _____

Checked by: _____

FINAL SYSTEM DOCUMENTATION

INVENTORY MANAGEMENT SYSTEM

Prepared by: _____

Grade/Section: _____

Subject: Java Programming

Instructor: _____

Date Submitted: _____

1. Introduction

The Inventory Management System is a Java console application for monitoring a small set of products, their prices, and available quantities.

2. Objectives

- To display product names, prices, and stock.
- To record stock added to a product.
- To warn the user when a product has low stock.

3. Scope and Limitations

Scope: The system contains sample products and supports viewing inventory, adding stock, and checking the current stock status.

Limitations: Products are predefined and data is kept only while the program is running. It does not process sales, suppliers, or database storage.

4. System Requirements

Hardware: computer or laptop. Software: Java JDK and IntelliJ IDEA or another Java IDE.

5. System Design

Algorithm: Start → Show inventory menu → View stock, add stock, or exit → Check product number and quantity → Update inventory → Repeat → End.

Program structure: The program imports Scanner, declares the required variables or arrays, displays a menu or prompts, processes input, validates user choices, and prints the result.

6. Source Code

```
import java.util.Scanner;

public class InventoryManagementSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] products = {"Notebook", "Ballpen", "Folder", "Pencil"};
        double[] prices = {25.00, 10.00, 15.00, 8.00};
        int[] stock = {20, 50, 15, 30};
        int choice;
```

```
do {
    System.out.println("\n=== INVENTORY MANAGEMENT SYSTEM ===");
    System.out.println("1. View Inventory");
    System.out.println("2. Add Stock");
    System.out.println("3. Exit");
    System.out.print("Choose: ");
    choice = sc.nextInt();

    switch (choice) {
        case 1:
            for (int i = 0; i < products.length; i++) {
                System.out.printf("%d. %s | PHP %.2f | Stock: %d%n",
                        i + 1, products[i], prices[i], stock[i]);
                if (stock[i] <= 10) {
                    System.out.println("   WARNING: Low stock!");
                }
            }
            break;
        case 2:
            System.out.print("Product number: ");
            int productNo = sc.nextInt();
            if (productNo >= 1 && productNo <= products.length) {
                System.out.print("Quantity to add: ");
                int quantity = sc.nextInt();
                if (quantity > 0) {
                    stock[productNo - 1] += quantity;
                    System.out.println("Stock updated.");
                } else {
                    System.out.println("Quantity must be greater than zero.");
                }
            } else {
                System.out.println("Invalid product number.");
            }
            break;
        case 3:
            System.out.println("Inventory program closed.");
            break;
        default:
            System.out.println("Invalid choice.");
        }
    } while (choice != 3);
    sc.close();
  }
}
```

7. Program Screenshots

Insert your own screenshots here after running the program in IntelliJ IDEA:

Screenshot 1: Program menu or opening screen

[Paste screenshot here]

Screenshot 2: Sample input and output

[Paste screenshot here]

Screenshot 3: Validation or final result

[Paste screenshot here]

8. Testing and Results

Test	Expected Result	Actual Result
Valid input	System processes input and shows the correct output.	To be completed after testing.
Invalid menu choice or number	System displays an error message.	To be completed after testing.
Exit option	Program closes normally.	To be completed after testing.

9. Conclusion and Recommendations

The Inventory Management System demonstrates how Java can be used to solve a basic information-processing task. It applies input and output, variables, conditional statements, and other beginner programming concepts. In the future, the system may be improved by adding data storage, a graphical user interface, stronger input validation, and additional functions.

Prepared by: _____

Checked by: _____

FINAL SYSTEM DOCUMENTATION

POINT OF SALE (POS) SYSTEM

Prepared by: _____

Grade/Section: _____

Subject: Java Programming

Instructor: _____

Date Submitted: _____

1. Introduction

The Point of Sale (POS) System is a simple Java program that lets a cashier select a product, enter a quantity, calculate the total price, accept payment, and calculate change.

2. Objectives

- To display a list of products and prices.
- To calculate the purchase total.
- To check payment and calculate the customer's change.

3. Scope and Limitations

Scope: The program handles one product purchase per transaction from a predefined list. It displays a simple receipt summary.

Limitations: This is a classroom demonstration only. It does not connect to a payment terminal, print a physical receipt, save sales, or calculate taxes and discounts.

4. System Requirements

Hardware: computer or laptop. Software: Java JDK and IntelliJ IDEA or another Java IDE.

5. System Design

Algorithm: Start → Show products → Select product and quantity → Calculate total → Enter payment → Check payment → Display total and change → End.

Program structure: The program imports Scanner, declares the required variables or arrays, displays a menu or prompts, processes input, validates user choices, and prints the result.

6. Source Code

```
import java.util.Scanner;

public class PointOfSaleSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] products = {"Notebook", "Ballpen", "Folder", "Pencil"};
        double[] prices = {25.00, 10.00, 15.00, 8.00};

        System.out.println("=== POINT OF SALE SYSTEM ===");
        for (int i = 0; i < products.length; i++) {
```

```
        System.out.printf("%d. %s - PHP %.2f%n", i + 1, products[i], prices[i]);
    }

    System.out.print("Select product number: ");
    int choice = sc.nextInt();

    if (choice >= 1 && choice <= products.length) {
        System.out.print("Enter quantity: ");
        int quantity = sc.nextInt();

        if (quantity > 0) {
            double total = prices[choice - 1] * quantity;
            System.out.printf("Product: %s%n", products[choice - 1]);
            System.out.printf("Total: PHP %.2f%n", total);
            System.out.print("Enter payment: PHP ");
            double payment = sc.nextDouble();

            if (payment >= total) {
                double change = payment - total;
                System.out.println("\n=== RECEIPT ===");
                System.out.println("Item: " + products[choice - 1]);
                System.out.println("Quantity: " + quantity);
                System.out.printf("Total: PHP %.2f%n", total);
                System.out.printf("Payment: PHP %.2f%n", payment);
                System.out.printf("Change: PHP %.2f%n", change);
            } else {
                System.out.println("Insufficient payment. Transaction not completed.");
            }
        } else {
            System.out.println("Quantity must be greater than zero.");
        }
    } else {
        System.out.println("Invalid product number.");
    }
    sc.close();
  }
}
```

7. Program Screenshots

Insert your own screenshots here after running the program in IntelliJ IDEA:

Screenshot 1: Program menu or opening screen

[Paste screenshot here]

Screenshot 2: Sample input and output

[Paste screenshot here]

Screenshot 3: Validation or final result

[Paste screenshot here]

8. Testing and Results

Test	Expected Result	Actual Result
Valid input	System processes input and shows the correct	To be completed after testing.

	output.	
Invalid menu choice or number	System displays an error message.	To be completed after testing.
Exit option	Program closes normally.	To be completed after testing.

9. Conclusion and Recommendations

The Point Of Sale (Pos) System demonstrates how Java can be used to solve a basic information-processing task. It applies input and output, variables, conditional statements, and other beginner programming concepts. In the future, the system may be improved by adding data storage, a graphical user interface, stronger input validation, and additional functions.

Prepared by: _____

Checked by: _____

FINAL SYSTEM DOCUMENTATION

STUDENT GRADE MANAGEMENT SYSTEM

Prepared by: _____

Grade/Section: _____

Subject: Java Programming

Instructor: _____

Date Submitted: _____

1. Introduction

The Student Grade Management System is a Java console application that accepts a student's name and grades, computes the average, and displays a simple academic result.

2. Objectives

- To input a student's name and three subject grades.
- To calculate the average grade.
- To display a pass or needs-improvement result using a condition.

3. Scope and Limitations

Scope: The program computes the average of three grades for one student and displays the result based on a passing average of 75.

Limitations: The program handles one student at a time and does not save results to a file or database. The passing threshold is a sample value and should be changed if the teacher requires a different grading policy.

4. System Requirements

Hardware: computer or laptop. Software: Java JDK and IntelliJ IDEA or another Java IDE.

5. System Design

Algorithm: Start → Enter name and three grades → Validate grades → Compute average → Determine result → Display summary → End.

Program structure: The program imports Scanner, declares the required variables or arrays, displays a menu or prompts, processes input, validates user choices, and prints the result.

6. Source Code

```
import java.util.Scanner;

public class StudentGradeManagementSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
System.out.println("=== STUDENT GRADE MANAGEMENT SYSTEM ===");
System.out.print("Student name: ");
String name = sc.nextLine();

System.out.print("Grade in Subject 1 (0-100): ");
double grade1 = sc.nextDouble();
System.out.print("Grade in Subject 2 (0-100): ");
double grade2 = sc.nextDouble();
System.out.print("Grade in Subject 3 (0-100): ");
double grade3 = sc.nextDouble();

if (validGrade(grade1) && validGrade(grade2) && validGrade(grade3)) {
    double average = (grade1 + grade2 + grade3) / 3.0;
    String result;

    if (average >= 75) {
        result = "PASSED";
    } else {
        result = "NEEDS IMPROVEMENT";
    }

    System.out.println("\n=== GRADE REPORT ===");
    System.out.println("Student: " + name);
            System.out.printf("Average: %.2f%n", average);
            System.out.println("Result: " + result);
        } else {
            System.out.println("Invalid grade. Enter grades from 0 to 100.");
        }
        sc.close();
    }

    public static boolean validGrade(double grade) {
        return grade >= 0 && grade <= 100;
    }
}
```

7. Program Screenshots

Insert your own screenshots here after running the program in IntelliJ IDEA:

Screenshot 1: Program menu or opening screen

[Paste screenshot here]

Screenshot 2: Sample input and output

[Paste screenshot here]

Screenshot 3: Validation or final result

[Paste screenshot here]

8. Testing andResults

Test	Expected Result	Actual Result
Valid input	System processes input and shows the correct	To be completed after testing.

	output.	
Invalid menu choice or number	System displays an error message.	To be completed after testing.
Exit option	Program closes normally.	To be completed after testing.

9. Conclusion and Recommendations

The Student Grade Management System demonstrates how Java can be used to solve a basic information-processing task. It applies input and output, variables, conditional statements, and other beginner programming concepts. In the future, the system may be improved by adding data storage, a graphical user interface, stronger input validation, and additional functions.

Prepared by: _____

Checked by: _____